Background/instructions

Determining which Exclusive Economic Zones (EEZ) on the West Coast of the US are best suited to developing marine aquaculture for several species of oysters and a species of your choice. Suitable locations will be determined based on range of suitable sea surface temperature (SST) and depth values for the species.

Choose a species

- Red Abalone (https://www.sealifebase.ca/summary/Haliotis-rufescens.html)
- Depth: 0-24m
- Temperature: 8°C - 18°C

Workflow outline

Prepare data

```
getwd()
```

```
[1] "/Users/oMac/Desktop/meds_stuff/eds223 repos folder/eds223-homeworks/EEZs-for-Marine-Aquaculture/EEZs-for-Marine-Aquaculture"
```

```
# Load necessary packages
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tidyr' was built under R version 4.4.3
```

```
Warning: package 'purrr' was built under R version 4.4.3
```

```
Warning: package 'dplyr' was built under R version 4.4.3
```

```
Warning: package 'lubridate' was built under R version 4.4.3
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   4.0.2     v tibble    3.3.0
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```r
library(sf)
```

```
Warning: package 'sf' was built under R version 4.4.3
```

```
Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
library(terra)
```

```
terra 1.8.54
```

```
Attaching package: 'terra'
```

```
The following object is masked from 'package:tidyr':

    extract
```

```r
library(tmap)
library(kableExtra)
```

```
Attaching package: 'kableExtra'
```

```
The following object is masked from 'package:dplyr':

    group_rows
```

```r
library(spData)
library(spDataLarge)
library(geodata)
library(elevatr)
```

```
elevatr v0.99.0 NOTE: Version 0.99.0 of 'elevatr' uses 'sf' and 'terra'.  Use
of the 'sp', 'raster', and underlying 'rgdal' packages by 'elevatr' is being
deprecated; however, get_elev_raster continues to return a RasterLayer.  This
will be dropped in future versions, so please plan accordingly.
```

```
library(tigris)
```

```
To enable caching of data, set `options(tigris_use_cache = TRUE)`
in your R script or .Rprofile.

Attaching package: 'tigris'

The following object is masked from 'package:terra':

    blocks
```

```
library(here)
```

```
here() starts at /Users/oMac/Desktop/meds_stuff/eds223 repos folder/eds223-
homeworks/EEZs-for-Marine-Aquaculture/EEZs-for-Marine-Aquaculture
```

```
getwd()
```

```
[1] "/Users/oMac/Desktop/meds_stuff/eds223 repos folder/eds223-homeworks/EEZs-
for-Marine-Aquaculture/EEZs-for-Marine-Aquaculture"
```

```
# Read in the data
# SST files (rasters)
sst_2008 <- terra::rast(here("data", "average_annual_sst_2008.tif"))
sst_2009 <- terra::rast(here("data", "average_annual_sst_2009.tif"))
sst_2010 <- terra::rast(here("data", "average_annual_sst_2010.tif"))
sst_2011 <- terra::rast(here("data", "average_annual_sst_2011.tif"))
sst_2012 <- terra::rast(here("data", "average_annual_sst_2012.tif"))
# Depth file (raster)
depth <- terra::rast(here("data", "depth.tif"))
# EEZ file (shapefile)
wc_eez_sf <- read_sf(here("data", "wc_regions_clean.shp"))

# Combine SST rasters into a raster stack
sst_stack <- c(sst_2008, sst_2009, sst_2010, sst_2011, sst_2012)
```

```r
# Check the class of the new raster (sanity check)
class(sst_stack)
```

```
[1] "SpatRaster"
attr(,"package")
[1] "terra"
```

/Users/oMac/Desktop/meds_stuff/eds223 repos folder/eds223-homeworks/EEZs-for-Marine-Aquaculture/EEZs-for-Marine-Aquaculture/aquaculture_analysis.qmd ### Process data

```r
# Find mean SST
sst_mean <- mean(sst_stack, na.rm=TRUE)
sst_mean
```

```
class       : SpatRaster
size        : 480, 408, 1  (nrow, ncol, nlyr)
resolution  : 0.04166185, 0.04165702  (x, y)
extent      : -131.9848, -114.9867, 29.99305, 49.98842  (xmin, xmax, ymin, ymax)
coord. ref. : lon/lat WGS 84
source(s)   : memory
name        :    mean
min value   : 278.130
max value   : 306.045
```

```r
# Convert avg SST from K to C
sst_mean_c <- sst_mean - 273.15

# Crop depth raster to match extent of SST raster
depth_crop <- crop(depth, sst_mean_c)

# Resample depth to match resolution of SST data using  bilinear method approach
depth_resampled <- resample(depth_crop, sst_mean_c, method = "bilinear")

# Check that depth and SST match in resolution, extent, and CRS
crs(depth_resampled) == crs(sst_mean_c)
```

```
[1] FALSE
```

```r
# They do not match in CRS, so we need to match them
depth_proj <- project(depth_crop, sst_mean_c)
depth_resampled <- resample(depth_proj, sst_mean_c, method = "bilinear")
crs(depth_resampled) == crs(sst_mean_c)
```

```
[1] TRUE
```

```r
# They now match in CRS.  Check resolution and extent
ext(depth_resampled) == ext(sst_mean_c)
```

```
[1] TRUE
```

```r
res(depth_resampled) == res(sst_mean_c)
```

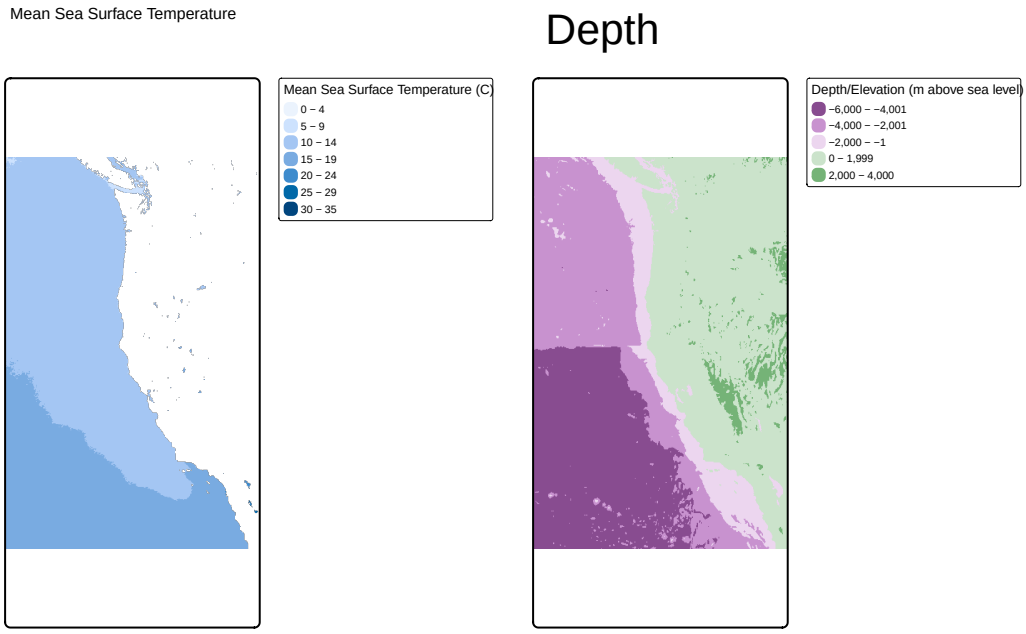
```
[1] TRUE TRUE
```

```r
# They match!

# Let's visualize what we have so far:
prelim_sst_map <- tm_shape(sst_mean_c) +
     tm_raster(col.legend = tm_legend(title = "Mean Sea Surface Temperature (C)")) +
     tm_title(text = "Mean Sea Surface Temperature")
prelim_depth_map <- tm_shape(depth_resampled) +
     tm_raster(col.legend = tm_legend(title = "Depth/Elevation (m above sea level)")) +
     tm_title(text = "Depth")
tmap_arrange(prelim_sst_map, prelim_depth_map, nrow = 1)
```

```
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
[scale] tm_raster:() the data variable assigned to 'col' contains positive and negative valu

[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```



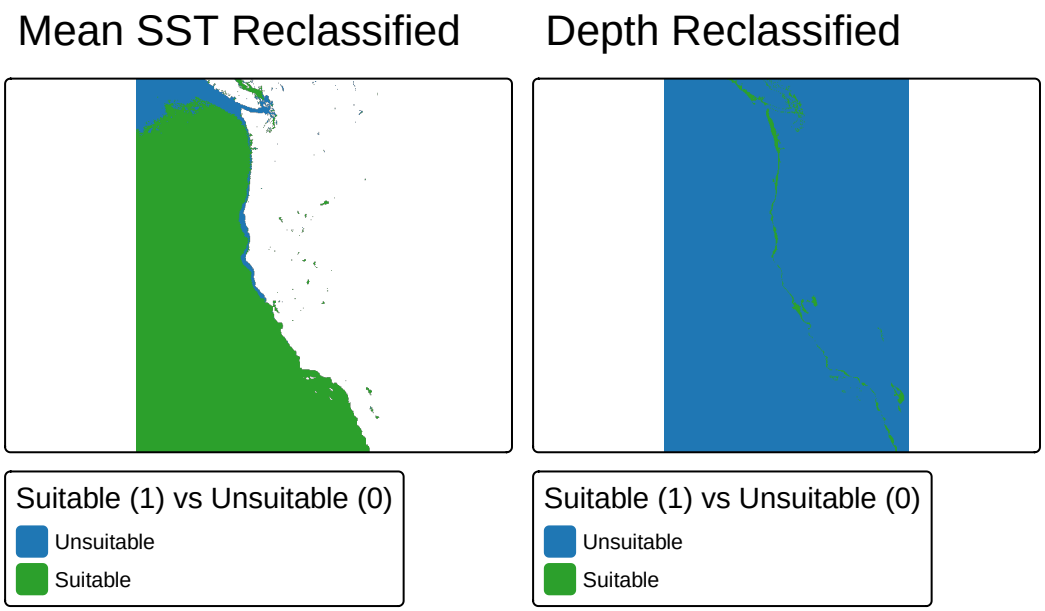
Find suitable locations

```
# Reclassify SST and depth data into locations that are suitable for oysters
# Set as "1" if depth is between 0-70m, otherwise set as zero
depth_matrix <- matrix(c(- Inf, -70, 0,
                         -70, 0, 1,
                         0, Inf, 0),
                         ncol = 3, byrow = TRUE)
depth_reclassified <- classify(depth_resampled, rcl = depth_matrix, include.lowest = TRUE)

# Do the same thing for SST, setting to zero if temp is not 11-30C
sst_matrix <- matrix(c(-Inf, 11, 0,
                         11, 30, 1,
                         30, Inf, 0),
                         ncol = 3, byrow = TRUE)
sst_reclassified <- classify(sst_mean_c, rcl = sst_matrix, include.lowest = TRUE)

# Let's look at what we have now:
reclass_sst_map <- tm_shape(sst_reclassified) +
     tm_raster(col.scale = tm_scale_categorical(labels = c("Unsuitable", "Suitable")),
     col.legend = tm_legend(title = "Suitable (1) vs Unsuitable (0)")) +
```

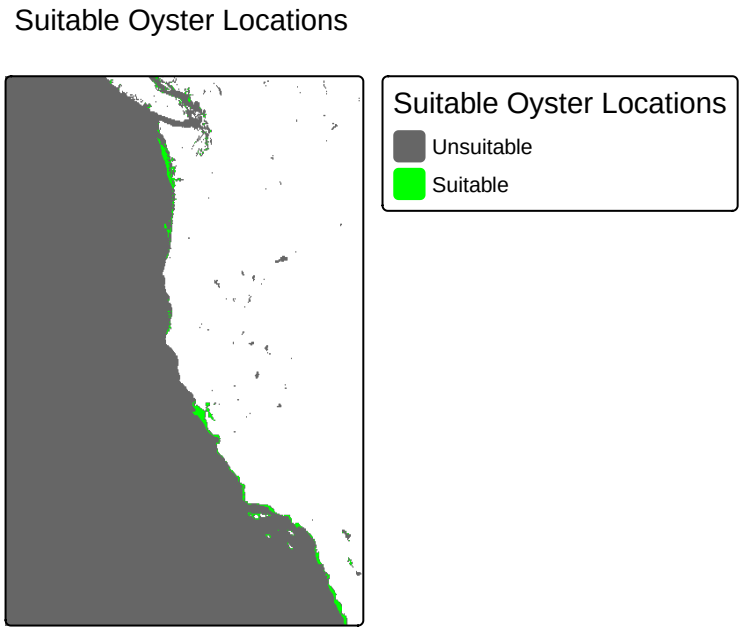
```
     tm_title(text = "Mean SST Reclassified")
reclass_depth_map <- tm_shape(depth_reclassified) +
     tm_raster(col.scale = tm_scale_categorical(labels = c("Unsuitable", "Suitable")),
     col.legend = tm_legend(title = "Suitable (1) vs Unsuitable (0)")) +
     tm_title(text = "Depth Reclassified")
tmap_arrange(reclass_sst_map, reclass_depth_map, nrow = 1)
```




```
# Find locations that satisfy both SST and depth conditions, using "lapp"
suitable_oyster <- lapp(c(sst_reclassified, depth_reclassified), fun = function(x, y) x*y)

# Map what we have now (the areas that satisfy the depth and temp requirements for oysters)
satisfy_map <- tm_shape(suitable_oyster) +
     tm_raster(col.scale = tm_scale_categorical(labels = c("Unsuitable", "Suitable"), values
          col.legend = tm_legend(title = "Suitable Oyster Locations")) +
     tm_title(text = "Suitable Oyster Locations")
satisfy_map
```

```
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```



Determine the most suitable EEZs

```
# Select suitable cells within West Coast EEZs.  We will need to overlay the suitability map

# First, we need to make sure the CRS match
crs(wc_eez_sf) == crs(suitable_oyster)
```

```
[1] FALSE
```

```
# CRS do NOT match
wc_eez_sf_matched <- st_transform(wc_eez_sf, crs(suitable_oyster))
crs(wc_eez_sf_matched) == crs(suitable_oyster)
```

```
[1] TRUE
```

```
# Next, find area of grid cells
cell_area <- cellSize(suitable_oyster, unit = "km")
suitable_area <- cell_area * suitable_oyster
```

```r
# Rasterize wc_eez_sf, so that both EEZ data and suitable sites data are rasters
wc_eez_sf_matched$id <- seq_len(nrow(wc_eez_sf_matched))
eez_raster <- rasterize(wc_eez_sf_matched, suitable_oyster, field = "id")

# Find the total suitable area within each EEZ
area_in_eez <- zonal(suitable_area, eez_raster, fun="sum", na.rm = TRUE)
names(area_in_eez) <- c("id", "suitable_area_km2")

# Create a final dataset of EEZs by area that is suitable for oysters
eez_fin <- wc_eez_sf_matched |>
     mutate(id = row_number()) |>
     left_join(area_in_eez, by = "id") |>
     mutate(suitable_area_km2 = coalesce(suitable_area_km2, 0)) |>
     arrange(desc(suitable_area_km2)) |>
     mutate(rank = row_number())
eez_fin
```

```
Simple feature collection with 5 features and 8 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -129.1635 ymin: 30.542 xmax: -117.097 ymax: 49.00031
Geodetic CRS:  WGS 84
# A tibble: 5 x 9
  rgn            rgn_key area_m2 rgn_id area_km2                 geometry    id
* <chr>          <chr>     <dbl>  <int>    <dbl>        <MULTIPOLYGON [°]> <dbl>
1 Central Calif~ CA-C    2.03e11      3  202738. (((-122.9928 38.28022, -
~     3
2 Southern Cali~ CA-S    2.07e11      4  206861. (((-120.6505 34.97431, -
~     4
3 Washington     WA      6.69e10      5   66898. (((-122.7675 49.00031, -
~     5
4 Oregon         OR      1.80e11      1  179994. (((-123.4318 46.23363, -
~     1
5 Northern Cali~ CA-N    1.64e11      2  164379. (((-124.2102 41.99843, -
~     2
# i 2 more variables: suitable_area_km2 <dbl>, rank <int>
```

Function

Wrap all of the code inside a function so that it is reproducible for any species, provided that we know the depth and temperature requirements for that species.

```r
# This version of the function is further simplified so that the only input is the species o

# Create pre-filled oyster and red abalone dataframes first (can do this easily for any speci
oyster <- data.frame(
     name = "Oyster",
     min_sst = 11,
     max_sst = 30,
     min_depth = 0,
     max_depth = 70
)
red_abalone <- data.frame(
     name = "Red Abalone",
     min_sst = 8,
     max_sst = 18,
     min_depth = 0,
     max_depth = 24
)


# Now create the function, where the sole argument is the dataframe for species of interest
simplified_aquaculture_analysis <- function(species){
     # read in data
     sst_2008 <- terra::rast(here("data", "average_annual_sst_2008.tif"))
     sst_2009 <- terra::rast(here("data", "average_annual_sst_2009.tif"))
     sst_2010 <- terra::rast(here("data", "average_annual_sst_2010.tif"))
     sst_2011 <- terra::rast(here("data", "average_annual_sst_2011.tif"))
     sst_2012 <- terra::rast(here("data", "average_annual_sst_2012.tif"))
     # Depth file (raster)
     depth <- terra::rast(here("data", "depth.tif"))
     # EEZ file (shapefile)
     wc_eez_sf <- read_sf(here("data", "wc_regions_clean.shp"))

     # Clean/crop/reshape data
     sst_stack <- c(sst_2008, sst_2009, sst_2010, sst_2011, sst_2012)
     sst_mean_c <- mean(sst_stack, na.rm=TRUE) - 273.15
     depth_crop <- crop(depth, sst_mean_c)
     depth_proj <- project(depth_crop, sst_mean_c)
     depth_resampled <- resample(depth_proj, sst_mean_c, method = "bilinear")

     # Filter for suitable locations
     depth_matrix <- matrix(c(- Inf, -(species$max_depth), 0,
                              -(species$max_depth), -(species$min_depth), 1,
```

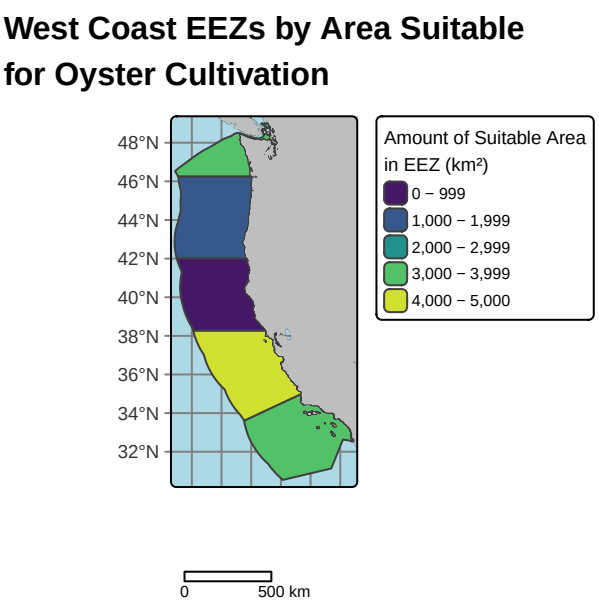
```
                          -(species$min_depth), Inf, 0),
                          ncol = 3, byrow = TRUE)
    depth_reclassified <- classify(depth_resampled, rcl = depth_matrix, include.lowest = TRU
    sst_matrix <- matrix(c(-Inf, species$min_sst, 0,
                          species$min_sst, species$max_sst, 1,
                          species$max_sst, Inf, 0),
                          ncol = 3, byrow = TRUE)
    sst_reclassified <- classify(sst_mean_c, rcl = sst_matrix, include.lowest = TRUE)
    suitable_spots <- lapp(c(sst_reclassified, depth_reclassified), fun = function(x, y) x*y

    # Calculate suitable area by EEZ
    wc_eez_sf_matched <- st_transform(wc_eez_sf, crs(suitable_spots))
    crs(wc_eez_sf_matched) == crs(suitable_spots)
    cell_area <- cellSize(suitable_spots, unit = "km")
    suitable_area <- cell_area * suitable_spots
    wc_eez_sf_matched$id <- seq_len(nrow(wc_eez_sf_matched))
    eez_raster <- rasterize(wc_eez_sf_matched, suitable_spots, field = "id")
    area_in_eez <- zonal(suitable_area, eez_raster, fun="sum", na.rm = TRUE)
    names(area_in_eez) <- c("id", "suitable_area_km2")
    eez_fin <- wc_eez_sf_matched |>
         mutate(id = row_number()) |>
         left_join(area_in_eez, by = "id") |>
         mutate(suitable_area_km2 = coalesce(suitable_area_km2, 0)) |>
         arrange(desc(suitable_area_km2)) |>
         mutate(rank = row_number())

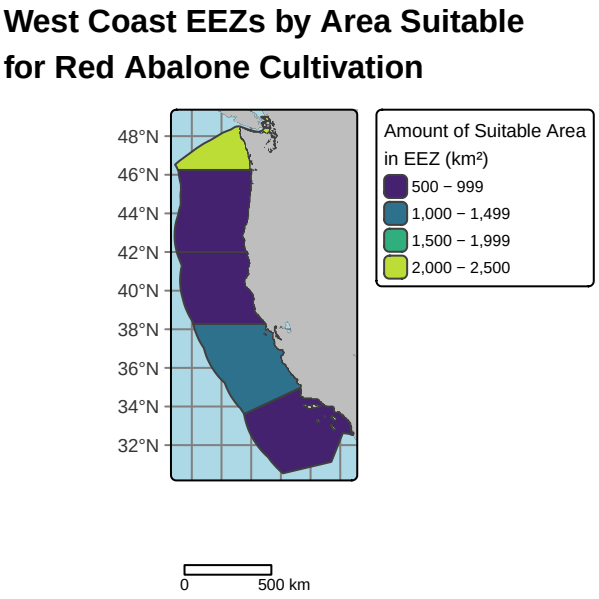
    # Map suitability
    land <- ifel(depth_resampled > 0, 1, NA)
    suit_map <- tm_shape(eez_fin) +
         tm_graticules() +
         tm_polygons("suitable_area_km2", fill.scale =
              tm_scale_intervals(values = "viridis"),
              fill.legend = tm_legend(
                   title = "Amount of Suitable Area\nin EEZ (km²)",
                   title.size = 0.6,
                   text.size = 0.5))  +
         tm_title(
              paste0("West Coast EEZs by Area Suitable\nfor ", species$name, " Cultivation")
              size = 1,
              fontface = "bold") +
         tm_scalebar(position = tm_pos_out("center", "bottom")) +
         tm_shape(land) +
```

```
              tm_raster(col.scale = tm_scale_categorical(values = "grey"),
            col.legend = tm_legend_hide()) +
          tm_layout(bg.color = "lightblue") +
          tm_options(component.autoscale = FALSE)
      suit_map
}

simplified_aquaculture_analysis(oyster)
```




```
simplified_aquaculture_analysis(red_abalone)
```




```
# This version of the function takes minimum and maximum depth and SST as arugments in additi
aquaculture_analysis <- function(min_sst, max_sst, min_depth, max_depth, species){
     # Read in data
     sst_2008 <- terra::rast(here("data", "average_annual_sst_2008.tif"))
     sst_2009 <- terra::rast(here("data", "average_annual_sst_2009.tif"))
     sst_2010 <- terra::rast(here("data", "average_annual_sst_2010.tif"))
     sst_2011 <- terra::rast(here("data", "average_annual_sst_2011.tif"))
     sst_2012 <- terra::rast(here("data", "average_annual_sst_2012.tif"))
     # Depth file (raster)
     depth <- terra::rast(here("data", "depth.tif"))
     # EEZ file (shapefile)
     wc_eez_sf <- read_sf(here("data", "wc_regions_clean.shp"))

     # Clean/crop/reshape data
     sst_stack <- c(sst_2008, sst_2009, sst_2010, sst_2011, sst_2012)
     sst_mean_c <- mean(sst_stack, na.rm=TRUE) - 273.15
     depth_crop <- crop(depth, sst_mean_c)
     depth_proj <- project(depth_crop, sst_mean_c)
     depth_resampled <- resample(depth_proj, sst_mean_c, method = "bilinear")

     # Filter for suitable locations
     depth_matrix <- matrix(c(- Inf, -(max_depth), 0,
                              -(max_depth), -(min_depth), 1,
```

```
                          -(min_depth), Inf, 0),
                          ncol = 3, byrow = TRUE)
depth_reclassified <- classify(depth_resampled, rcl = depth_matrix, include.lowest = TRU
sst_matrix <- matrix(c(-Inf, min_sst, 0,
                          min_sst, max_sst, 1,
                          max_sst, Inf, 0),
                          ncol = 3, byrow = TRUE)
sst_reclassified <- classify(sst_mean_c, rcl = sst_matrix, include.lowest = TRUE)
suitable_spots <- lapp(c(sst_reclassified, depth_reclassified), fun = function(x, y) x*y

# Calculate suitable area by EEZ
wc_eez_sf_matched <- st_transform(wc_eez_sf, crs(suitable_spots))
crs(wc_eez_sf_matched) == crs(suitable_spots)
cell_area <- cellSize(suitable_spots, unit = "km")
suitable_area <- cell_area * suitable_spots
wc_eez_sf_matched$id <- seq_len(nrow(wc_eez_sf_matched))
eez_raster <- rasterize(wc_eez_sf_matched, suitable_spots, field = "id")
area_in_eez <- zonal(suitable_area, eez_raster, fun="sum", na.rm = TRUE)
names(area_in_eez) <- c("id", "suitable_area_km2")
eez_fin <- wc_eez_sf_matched |>
     mutate(id = row_number()) |>
     left_join(area_in_eez, by = "id") |>
     mutate(suitable_area_km2 = coalesce(suitable_area_km2, 0)) |>
     arrange(desc(suitable_area_km2)) |>
     mutate(rank = row_number())

# Map suitability
land <- ifel(depth_resampled > 0, 1, NA)
suit_map <- tm_shape(eez_fin) +
     tm_graticules() +
     tm_polygons("suitable_area_km2", fill.scale =
          tm_scale_intervals(values = "viridis"),
          fill.legend = tm_legend(
               title = "Amount of Suitable Area\nin EEZ (km²)",
               title.size = 0.6,
               text.size = 0.5))  +
     tm_title(
          paste0("West Coast EEZs by Area Suitable\nfor ", species, " Cultivation"),
          size = 1,
          fontface = "bold") +
     tm_scalebar(position = tm_pos_out("center", "bottom")) +
     tm_shape(land) +
```

```
                tm_raster(col.scale = tm_scale_categorical(values = "grey"),
              col.legend = tm_legend_hide()) +
          tm_layout(bg.color = "lightblue") +
          tm_options(component.autoscale = FALSE)
      suit_map
}

aquaculture_analysis(8, 18, 0, 24, "Oyster")
```

